

Deepwoken Build Analyzer — 사용자 매뉴얼

멀티모달 AI(Gemini 2.5) & 서브 에이전트 기반 빌드 분석 및 RAG 추천 시스템

1. 시스템 개요 (System Overview)

Deepwoken Build Analyzer는 Roblox의 하드코어 액션 RPG Deepwoken의 방대한 빌드 데이터를 유튜브 영상(화면 UI + 음성 해설) 및 웹사이트(위키/빌드 플레너/포럼)로부터 자동 추출하여 정형 JSON 및 고가독성 Markdown 문서로 구조화하고, ChromaDB 벡터 데이터베이스(RAG)를 통해 사용자 맞춤형 빌드를 실시간으로 추천해주는 AI 솔루션입니다.

2. 빠른 시작 & 환경 설정 (Setup Guide)

단계	실행 명령어 및 작업 내용	설명
1. 의존성 설치	<code>pip install -r requirements.txt</code>	yt-dlp, google-generativeai, chromadb, rich 등 설치
2. API 키 등록	<code>.env</code> 파일 생성 후 <code>GEMINI_API_KEY=AlzaSy...</code> 입력	Google AI Studio 발급 키 등록
3. 설정 커스텀	<code>config/settings.yaml</code> (모델, 경로 등 설정)	Gemini 모델(Pro/Flash), ChromaDB 경로 지정

3. 핵심 CLI 명령어 가이드 (CLI Reference)

모드 (Command)	사용 예시 (Example)	핵심 기능 및 동작
analyze (단일 영상 분석)	<code>python main.py analyze "https://youtu.be/..."</code>	유튜브 고화질 영상 다운로드 → Gemini Files API 멀티모달 분석 → JSON/MD 생성 → ChromaDB 자동 인덱싱
web (웹 서브에이전트)	<code>python main.py web "https://deepwoken.co/..."</code>	웹페이지 스크래핑 → 2개 서브 에이전트(Build/Context) 병렬 파싱 → 교차 검증 → RAG 자동 등록
batch (대량 일괄 분석)	<code>python main.py batch "PLAYLIST_URL" --playlist</code>	재생목록 내 모든 영상 순차 분석 (Rate Limit 대응 딜레이 및 오류 복구 내장)
index (RAG 재인덱싱)	<code>python main.py index</code>	data/analysis 및 knowledge_base의 모든 빌드를 ChromaDB에 일괄 재색인
chat (AI 빌드 추천)	<code>python main.py chat</code>	RAG 벡터 검색 기반 대화형 어드바이저 실행 (스탯, 무기, 탠런트, 콤보 추천)

Tip: 영상 파일은 2GB 초과 시 자동으로 720p/480p로 다운스케일링되며, 분석 완료 후 로컬 디스크 절약을 위해 임시 영상이 자동 삭제됩니다.

Deepwoken Build Analyzer — 실전 작동 예시 및 동작 원리

서브 에이전트 협업 메커니즘 & RAG 기반 AI 빌드 추천 시나리오

1. 멀티 서브 에이전트 동작 원리 (Internal Architecture)

단계	담당 서브 에이전트	상세 동작 원리 및 역할
1단계: 수집	WebScaperAgent (웹) / VideoCollector(영상)	HTML 파싱(태그/표/메타 추출) 또는 yt-dlp 최고화질 다운로드 및 메타데이터 추출
2단계: 분석	BuildParser & ContextParser (병렬)	- BuildParser: 스탯(STR/FORT/AGI 등), 속성, Oath, 탠트, 만트라, SoO 추출 - ContextParser: PvP/PvE 분류, 난이도, 제작자 의도, 장단점, 콤보 사이클 추출
3단계: 검증	CrossValidatorAgent	서브 에이전트 추출 결과 병합, Deepwoken 인게임 룰셋 검증(스탯 범위, Oath 정규화) 및 정제
4단계: 지식화	BuildStructurer & KnowledgeBuilder	data/analysis/ID.json 및 data/knowledge_base/ID.md 생성 후 ChromaDB 벡터 DB 임베딩
5단계: 추천	BuildAdvisor (챗봇)	사용자 자연어 쿼리 벡터 검색 → 관련 상위 빌드 컨텍스트 결합 → Gemini 맞춤형 조언 생성

2. 실전 작동 예시 (Step-by-Step Walkthrough)

[시나리오 1] 웹사이트 빌드 분석 실행

```
> python main.py web "https://deepwoken.co/builder?id=sample_build"
[1/5] WebScaper: HTML 및 테이블 파싱 완료
[2/5] BuildParser & ContextParser: 병렬 AI 분석 수행 완료
[3/5] CrossValidator: JSON 스키마 및 Oath('Jetstriker') 정규화 완료
[4/5] ChromaDB: 'Thundercall Speedster' 벡터 인덱싱 완료 (소요: 3.2초)
```

[시나리오 2] RAG 기반 AI 빌드 추천 챗봇 대화

```
> python main.py chat
질문: "포티튜드 50 이상이고 빠른 기동성을 가진 Thundercall PvP 빌드 추천해줘"
어드바이저 응답:
추천 빌드: Thundercall Jetstriker Speedster (PvP / Advanced)
• 스탯 분배: STR 80, FORT 50 (Exoskeleton 확보), AGI 40, WIL 25, Thundercall 80
• 핵심 Oath & 탠트: Jetstriker / Showstopper, Exoskeleton, Lightning Cloak
• 추천 콤보: Slide 대시 진입 → M1 평타 2타 → Lightning Cloak 발동 후 Mantra 연계
• 출처: deepwoken.co / 영상 링크 제공
```

3. 운영 및 최적화 팁 (Pro Tips)

- 비용 최적화: config/settings.yaml에서 gemini.model을 gemini-2.5-flash로 설정하면 비용과 속도를 극대화할 수 있습니다.
- 데이터 수정: data/knowledge_base/*.md 파일을 직접 편집한 후 python main.py index를 실행하면 즉시 지식 베이스에 반영됩니다.